Chapter 7. Packaging, Releasing, and Daily Development

This chapter is about how free software projects package and release their software, and how overall development patterns organize around those goals.

A major difference between open source projects and proprietary ones is the lack of centralized control over the development team. When a new release is being prepared, this difference is especially stark: a corporation can ask its entire development team to focus on an upcoming release, putting aside new feature development and non-critical bug fixing until the release is done. Volunteer groups are not so monolithic. People work on the project for all sorts of reasons, and those not interested in helping with a given release still want to continue regular development work while the release is going on. Because development doesn't stop, open source release processes tend to take longer, but be less disruptive, than commercial release processes. It's a bit like highway repair. There are two ways to fix a road: you can shut it down completely, so that a repair crew can swarm all over it at full capacity until the problem is solved, or you can work on a couple of lanes at a time, while leaving the others open to traffic. The first way is very efficient *for the repair crew*, but not for anyone else—the road is entirely shut down until the job is done. The second way involves much more time and trouble for the repair crew (now they have to work with fewer people and less equipment, in cramped conditions, with flaggers to slow and direct traffic, etc.), but at least the road remains useable, albeit not at full capacity.

Open source projects tend to work the second way. In fact, for a mature piece of software with several different release lines being maintained simultaneously, the project is sort of in a permanent state of minor road repair. There are always a couple of lanes closed; a consistent but low level of background inconvenience is always being tolerated by the development group as a whole, so that releases get made on a regular schedule.

The model that makes this possible generalizes to more than just releases. It's the principle of parallelizing tasks that are not mutually interdependent—a principle that is by no means unique to open source development, of course, but one which open source projects implement in their own particular way. They cannot afford to annoy either the roadwork crew or the regular traffic too much, but they also cannot afford to have people dedicated to standing by the orange cones and flagging traffic along. Thus they gravitate toward processes that have flat, constant levels of administrative overhead, rather than peaks and valleys. Volunteers are generally willing to work with small but consistent amounts of inconvenience; the predictability allows them to come and go without worrying about whether their schedule will clash with what's happening in the project. But if the project were subject to a master schedule in which some activities excluded other activities, the result would be a lot of developers sitting idle a lot of the time—which would be not only inefficient but boring, and therefore dangerous, in that a bored developer is likely to soon be an ex-developer.

Release work is usually the most noticeable non-development task that happens in parallel with development, so the methods described in the following sections are geared mostly toward enabling releases. However, note that they also apply to other parallelizable tasks, such as translations and internationalization, broad API changes made gradually across the entire code base, etc.

Release Numbering

Before we talk about how to make a release, let's look at how to name releases, which requires knowing what releases actually mean to users. A release means that:

- Old bugs have been fixed. This is probably the one thing users can count on being true of every release.

- New bugs have been added. This too can usually be counted on, except sometimes in the case of security releases or other one-offs (see the section called “Security Releases” later in this chapter).
- New features may have been added.
- New configuration options may have been added, or the meanings of old options may have changed subtly. The installation procedures may have changed slightly since the last release too, though one always hopes not.
- Incompatible changes may have been introduced, such that the data formats used by older versions of the software are no longer useable without undergoing some sort of (possibly manual) one-way conversion step.

As you can see, not all of these are good things. This is why experienced users approach new releases with some trepidation, especially when the software is mature and was already mostly doing what they wanted (or thought they wanted). Even the arrival of new features is a mixed blessing, in that it may mean the software will now behave in unexpected ways.

The purpose of release numbering, therefore, is twofold: obviously the numbers should unambiguously communicate the ordering of releases (i.e., by looking at any two releases' numbers, one can know which came later), but also they should indicate as compactly as possible the degree and nature of the changes in the release.

All that in a number? Well, more or less, yes. Release numbering strategies are one of the oldest bikeshed discussions around (see the section called “The Softer the Topic, the Longer the Debate” in Chapter 6, *Communications*), and the world is unlikely to settle on a single, complete standard anytime soon. However, a few good strategies have emerged, along with one universally agreed-on principle: *be consistent*. Pick a numbering scheme, document it, and stick with it. Your users will thank you.

Release Number Components

This section describes the formal conventions of release numbering in detail, and assumes very little prior knowledge. It is intended mainly as a reference. If you're already familiar with these conventions, you can skip this section.

Release numbers are groups of digits separated by dots:

```
Scanley 2.3
Singer 5.11.4
```

...and so on. The dots are *not* decimal points, they are merely separators; "5.3.9" would be followed by "5.3.10". A few projects have occasionally hinted otherwise, most famously the Linux kernel with its "0.95", "0.96"... "0.99" sequence leading up to Linux 1.0, but the convention that the dots are not decimals is now firmly established and should be considered a standard. There is no limit to the number of components (digit portions containing no dots), but most projects do not go beyond three or four. The reasons why will become clear later.

In addition to the numeric components, projects sometimes tack on a descriptive label such as "Alpha" or "Beta" (see Alpha and Beta), for example:

```
Scanley 2.3.0 (Alpha)
Singer 5.11.4 (Beta)
```

An Alpha or Beta qualifier means that this release *precedes* a future release that will have the same number without the qualifier. Thus, "2.3.0 (Alpha)" leads eventually to "2.3.0". In order to allow several such candidate releases in a row, the qualifiers themselves can have meta-qualifiers. For example, here is a series of releases in the order that they would be made available to the public:

Scanley 2.3.0 (Alpha 1)
Scanley 2.3.0 (Alpha 2)
Scanley 2.3.0 (Beta 1)
Scanley 2.3.0 (Beta 2)
Scanley 2.3.0 (Beta 3)
Scanley 2.3.0

Notice that when it has the "Alpha" qualifier, Scanley "2.3" is written as "2.3.0". The two numbers are equivalent—trailing all-zero components can always be dropped for brevity—but when a qualifier is present, brevity is out the window anyway, so one might as well go for completeness instead.

Other qualifiers in semi-regular use include "Stable", "Unstable", "Development", and "RC" (for "Release Candidate"). The most widely used ones are still "Alpha" and "Beta", with "RC" running a close third place, but note that "RC" always includes a numeric meta-qualifier. That is, you don't release "Scanley 2.3.0 (RC)", you release "Scanley 2.3.0 (RC 1)", followed by RC2, etc.

Those three labels, "Alpha", "Beta", and "RC", are pretty widely known now, and I don't recommend using any of the others, even though the others might at first glance seem like better choices because they are normal words, not jargon. But people who install software from releases are already familiar with the big three, and there's no reason to do things gratuitously differently from the way everyone else does them.

Although the dots in release numbers are not decimal points, they do indicate place-value significance. All "0.X.Y" releases precede "1.0" (which is equivalent to "1.0.0", of course). "3.14.158" immediately precedes "3.14.159", and non-immediately precedes "3.14.160" as well as "3.15.anything", and so.

A consistent release numbering policy enables a user to look at two release numbers for the same piece of software and tell, just from the numbers, the important differences between those two releases. In a typical three-component system, the first component is the *major number*, the second is the *minor number*, and the third is the *micro number*. For example, release "2.10.17" is the seventeenth micro release in the tenth minor release line within the second major release series. The words "line" and "series" are used informally here, but they mean what one would expect. A major series is simply all the releases that share the same major number, and a minor series (or minor line) consists of all the releases that share the same minor *and* major number. That is, "2.4.0" and "3.4.1" are not in the same minor series, even though they both have "4" for their minor number; on the other hand, "2.4.0" and "2.4.2" are in the same minor line, though they are not adjacent if "2.4.1" was released between them.

The meanings of these numbers are exactly what you'd expect: an increment of the major number indicates that major changes happened; an increment of the minor number indicates minor changes; and an increment of the micro number indicates really trivial changes. Some projects add a fourth component, usually called the *patch number*, for especially fine-grained control over the differences between their releases (confusingly, other projects use "patch" as a synonym for "micro" in a three-component system). There are also projects that use the last component as a *build number*, incremented every time the software is built and representing no change other than that build. This helps the project link every bug report with a specific build, and is probably most useful when binary packages are the default method of distribution.

Although there are many different conventions for how many components to use, and what the components mean, the differences tend to be minor—you get a little leeway, but not a lot. The next two sections discuss some of the most widely used conventions.

The Simple Strategy

Most projects have rules about what kinds of changes are allowed into a release if one is only incrementing the micro number, different rules for the minor number, and still different ones for the major number. There is no set standard for these rules yet, but here I will describe a policy that has been used successfully by multiple projects. You may want to just adopt this policy in your own project, but even if you don't, it's still a good example of the kind of information release numbers should convey. This policy is adapted from the numbering system used by the APR project, see http://apr.apache.org/versioning.html.

1. Changes to the micro number only (that is, changes within the same minor line) must be both forward- and backward-compatible. That is, the changes should be bug fixes only, or very small enhancements to existing features. New features should not be introduced in a micro release.

2. Changes to the minor number (that is, within the same major line) must be backward-compatible, but not necessarily forward-compatible. It's normal to introduce new features in a minor release, but usually not too many new features at once.

3. Changes to the major number mark compatibility boundaries. A new major release can be forward- and backward-incompatible. A major release is expected to have new features, and may even have entire new feature sets.

What *backward-compatible* and *forward-compatible* mean, exactly, depends on what your software does, but in context they are usually not open to much interpretation. For example, if your project is a client/server application, then "backward-compatible" means that upgrading the server to 2.6.0 should not cause any existing 2.5.4 clients to lose functionality or behave differently than they did before (except for bugs that were fixed, of course). On the other hand, upgrading one of those clients to 2.6.0, along with the server, might make *new* functionality available for that client, functionality that 2.5.4 clients don't know how to take advantage of. If that happens, then the upgrade is *not* "forward-compatible": clearly you can't now downgrade that client back to 2.5.4 and keep all the functionality it had at 2.6.0, since some of that functionality was new in 2.6.0.

This is why micro releases are essentially for bug fixes only. They must remain compatible in both directions: if you upgrade from 2.5.3 to 2.5.4, then change your mind and downgrade back to 2.5.3, no functionality should be lost. Of course, the bugs fixed in 2.5.4 would reappear after the downgrade, but you wouldn't lose any features, except insofar as the restored bugs prevent the use of some existing features.

Client/server protocols are just one of many possible compatibility domains. Another is data formats: does the software write data to permanent storage? If so, the formats it reads and writes need to follow the compatibility guidelines promised by the release number policy. Version 2.6.0 needs to be able to read the files written by 2.5.4, but may silently upgrade the format to something that 2.5.4 cannot read, because the ability to downgrade is not required across a minor number boundary. If your project distributes code libraries for other programs to use, then APIs are a compatibility domain too: you must make sure that source and binary compatibility rules are spelled out in such a way that the informed user need never wonder whether or not it's safe to upgrade in place. She will be able to look at the numbers and know instantly.

In this system, you don't get a chance for a fresh start until you increment the major number. This can often be a real inconvenience: there may be features you wish to add, or protocols that you wish to redesign, that simply cannot be done while maintaining compatibility. There's no magic solution to this, except to try to design things in an extensible way in the first place (a topic easily worth its own book, and certainly outside the scope of this one). But publishing a release compatibility policy, and adhering to it, is an inescapable part of distributing software. One nasty surprise can alienate a lot of users. The policy just described is good partly because it's already quite widespread, but also because it's easy to explain and to remember, even for those not already familiar with it.

It is generally understood that these rules do not apply to pre-1.0 releases (although your release policy should probably state so explicitly, just to be clear). A project that is still in initial development can release 0.1, 0.2, 0.3, and so on in sequence, until it's ready for 1.0, and the differences between those releases can be arbitrarily large. Micro numbers in pre-1.0 releases are optional. Depending on the nature of your project and the differences between the releases, you might find it useful to have 0.1.0, 0.1.1, etc., or you might not. Conventions for pre-1.0 release numbers are fairly loose, mainly because people understand that strong compatibility constraints would hamper early development too much, and because early adopters tend to be forgiving anyway.

Remember that all these injunctions only apply to this particular three-component system. Your project could easily come up with a different three-component system, or even decide it doesn't need such fine granularity and use a two-component system instead. The important thing is to decide early, publish exactly what the components mean, and stick to it.

The Even/Odd Strategy

Some projects use the parity of the minor number component to indicate the stability of the software: even means stable, odd means unstable. This applies only to the minor number, not the major and micro numbers. Increments in the micro number still indicate bug fixes (no new features), and increments in the major number still indicate big changes, new feature sets, etc.

The advantage of the even/odd system, which has been used by the Linux kernel project among others, is that it offers a way to release new functionality for testing without subjecting production users to potentially unstable code. People can see from the numbers that "2.4.21" is okay to install on their live web server, but that "2.5.1" should probably stay confined to home workstation experiments. The development team handles the bug reports that come in from the unstable (odd-minor-numbered) series, and when things start to settle down after some number of micro releases in that series, they increment the minor number (thus making it even), reset the micro number back to "0", and release a presumably stable package.

This system preserves, or at least, does not conflict with, the compatibility guidelines given earlier. It simply overloads the minor number with some extra information. This forces the minor number to be incremented about twice as often as would otherwise be necessary, but there's no great harm in that. The even/odd system is probably best for projects that have very long release cycles, and which by their nature have a high proportion of conservative users who value stability above new features. It is not the only way to get new functionality tested in the wild, however. the section called “Stabilizing a Release” later in this chapter describes another, perhaps more common, method of releasing potentially unstable code to the public, marked so that people have an idea of the risk/benefit trade-offs immediately on seeing the release's name.

Release Branches

From a developer's point of view, a free software project is in a state of continuous release. Developers usually run the latest available code at all times, because they want to spot bugs, and because they follow the project closely enough to be able to stay away from currently unstable areas of the feature space. They often update their copy of the software every day, sometimes more than once a day, and when they check in a change, they can reasonably expect that every other developer will have it within 24 hours.

How, then, should the project make a formal release? Should it simply take a snapshot of the tree at a moment in time, package it up, and hand it to the world as, say, version "3.5.0"? Common sense says no. First, there may be no moment in time when the entire development tree is clean and ready for release. Newly-started features could be lying around in various states of completion. Someone might have checked in a major change to fix a bug, but the change could be controversial and under debate at the moment the snapshot is taken. If so, it wouldn't work to simply delay the snapshot until the debate ends, because another, unrelated debate could start in the meantime, and then you'd have wait for *that* one to end too. This process is not guaranteed to halt.

In any case, using full-tree snapshots for releases would interfere with ongoing development work, even if the tree could be put into a releasable state. Say this snapshot is going to be "3.5.0"; presumably, the next snapshot would be "3.5.1", and would contain mostly fixes for bugs found in the 3.5.0 release. But if both are snapshots from the same tree, what are the developers supposed to do in the time between the two releases? They can't be adding new features; the compatibility guidelines prevent that. But not everyone will be enthusiastic about fixing bugs in the 3.5.0 code. Some people may have new features they're trying to complete, and will become irate if they are forced to choose between sitting idle and working on things they're not interested in, just because the project's release processes demand that the development tree remain unnaturally quiescent.

The solution to these problems is to always use a *release branch*. A release branch is just a branch in the version control system (see *branch*), on which the code destined for this release can be isolated from mainline development. The concept of release branches is certainly not original to free software; many commercial development organizations use them too. However, in commercial environments, release branches are sometimes considered a luxury—a kind of formal "best practice" that can, in the heat of a major deadline, be dispensed with while everyone on the team scrambles to stabilize the main tree.

Release branches are pretty much required in open source projects, however. I have seen projects do releases without them, but it has always resulted in some developers sitting idle while others—usually a minority—work on getting the release out the door. The result is usually bad in several ways. First, overall development momentum is slowed. Second, the release is of poorer quality than it needed to be, because there were only a few people working on it, and they were hurrying to finish so everyone else could get back to work. Third, it divides the development team psychologically, by setting up a situation in which different types of work interfere with each other unnecessarily. The developers sitting idle would probably be happy to contribute *some* of their attention to a release branch, as long as that were a choice they could make according to their own schedules and interests. But without the branch, their choice becomes "Do I participate in the project today or not?" instead of "Do I work on the release today, or work on that new feature I've been developing in the mainline code?"

Mechanics of Release Branches

The exact mechanics of creating a release branch depend on your version control system, of course, but the general concepts are the same in most systems. A branch usually sprouts from another branch or from the trunk. Traditionally, the trunk is where mainline development goes on, unfettered by release constraints. The first release branch, the one leading to the "1.0" release, sprouts off the trunk. In CVS, the branch command would be something like this

```
$ cd trunk-working-copy
$ cvs tag -b RELEASE_1_0_X
```

or in Subversion, like this:

```
$ svn copy http://.../repos/trunk http://.../repos/branches/1.0.x
```

(All these examples assume a three-component release numbering system. While I can't show the exact commands for every version control system, I'll give examples in CVS and Subversion and hope that the corresponding commands in other systems can be deduced from those two.)

Notice that we created branch "1.0.x" (with a literal "x") instead of "1.0.0". This is because the same minor line—i.e., the same branch—will be used for all the micro releases in that line. The actual process of stabilizing the branch for release is covered in the section called “Stabilizing a Release” later in this chapter. Here we are concerned just with the interaction between the version control system and the release process. When the release branch is stabilized and ready, it is time to tag a snapshot from the branch:

```
$ cd RELEASE_1_0_X-working-copy
$ cvs tag RELEASE_1_0_0
```

or

```
$ svn copy http://.../repos/branches/1.0.x http://.../repos/tags/1.0.0
```

That tag now represents the exact state of the project's source tree in the 1.0.0 release (this is useful in case anyone ever needs to get an old version after the packaged distributions and binaries have been taken down). The next micro release in the same line is likewise prepared on the 1.0.x branch, and when it is ready, a tag is made for 1.0.1. Lather, rinse, repeat for 1.0.2, and so on. When it's time to start thinking about a 1.1.x release, make a new branch from trunk:

```
$ cd trunk-working-copy
$ cvs tag -b RELEASE_1_1_X
```

or

```
$ svn copy http://.../repos/trunk http://.../repos/branches/1.1.x
```

Maintenance can continue in parallel along both 1.0.x and 1.1.x, and releases can be made independently from both lines. In fact, it is not unusual to publish near-simultaneous releases from two different lines. The older series is recommended for more conservative site administrators, who may not want to make the big jump to (say) 1.1 without careful preparation. Meanwhile, more adventurous people usually take the most recent release on the highest line, to make sure they're getting the latest features, even at the risk of greater instability.

This is not the only release branch strategy, of course. In some circumstances it may not even be the best, though it's worked out pretty well for projects I've been involved in. Use any strategy that seems to work, but remember the main points: the purpose of a release branch is to isolate release work from the fluctuations of daily development, and to give the project a physical entity around which to organize its release process. That process is described in detail in the next section.

Stabilizing a Release

Stabilization is the process of getting a release branch into a releasable state; that is, of deciding which changes will be in the release, which will not, and shaping the branch content accordingly.

There's a lot of potential grief contained in that word, "deciding". The last-minute feature rush is a familiar phenomenon in collaborative software projects: as soon as developers see that a release is about to happen, they scramble to finish their current changes, in order not to miss the boat. This, of course, is the exact opposite of what you want at release time. It would be much better for people to work on features at a comfortable pace, and not worry too much about whether their changes make it into this release or the next one. The more changes one tries to cram into a release at the last minute, the more the code is destabilized, and (usually) the more new bugs are created.

Most software engineers agree in theory on rough criteria for what changes should be allowed into a release line during its stabilization period. Obviously, fixes for severe bugs can go in, especially for bugs without workarounds. Documentation updates are fine, as are fixes to error messages (except when they are considered part of the interface and must remain stable). Many projects also allow certain kinds of low-risk or non-core changes to go in during stabilization, and may have formal guidelines for measur-

ing risk. But no amount of formalization can obviate the need for human judgement. There will always be cases where the project simply has to make a decision about whether a given change can go into a release. The danger is that since each person wants to see their own favorite changes admitted into the release, then there will be plenty of people motivated to allow changes, and not enough people motivated to bar them.

Thus, the process of stabilizing a release is mostly about creating mechanisms for saying "no". The trick for open source projects, in particular, is to come up with ways of saying "no" that won't result in too many hurt feelings or disappointed developers, and also won't prevent deserving changes from getting into the release. There are many different ways to do this. It's pretty easy to design systems that satisfy these criteria, once the team has focused on them as the important criteria. Here I'll briefly describe two of the most popular systems, at the extreme ends of the spectrum, but don't let that discourage your project from being creative. Plenty of other arrangements are possible; these are just two that I've seen work in practice.

Dictatorship by Release Owner

The group agrees to let one person be the *release owner*. This person has final say over what changes make it into the release. Of course, it is normal and expected for there to be discussions and arguments, but in the end the group must grant the release owner sufficient authority to make final decisions. For this system to work, it is necessary to choose someone with the technical competence to understand all the changes, and the social standing and people skills to navigate the discussions leading up to the release without causing too many hurt feelings.

A common pattern is for the release owner to say "I don't think there's anything wrong with this change, but we haven't had enough time to test it yet, so it shouldn't go into this release." It helps a lot if the release owner has broad technical knowledge of the project, and can give reasons why the change could be potentially destabilizing (for example, its interactions with other parts of the software, or portability concerns). People will sometimes ask such decisions to be justified, or will argue that a change is not as risky as it looks. These conversations need not be confrontational, as long as the release owner is able to consider all the arguments objectively and not reflexively dig in his heels.

Note that the release owner need not be the same person as the project leader (in cases where there is a project leader at all; see the section called “Benevolent Dictators” in Chapter 4, *Social and Political Infrastructure*). In fact, sometimes it's good to make sure they're *not* the same person. The skills that make a good development leader are not necessarily the same as those that make a good release owner. In something as important as the release process, it may be wise to have someone provide a counterbalance to the project leader's judgement.

Contrast the release owner role with the less dictatorial role described in the section called “Release manager” later in this chapter.

Change Voting

At the opposite extreme from dictatorship by release owner, developers can simply vote on which changes to include in the release. However, since the most important function of release stabilization is to *exclude* changes, it's important to design the voting system in such a way that getting a change into the release involves positive action by multiple developers. Including a change should need more than just a simple majority (see the section called “Who Votes?” in Chapter 4, *Social and Political Infrastructure*). Otherwise, one vote for and none against a given change would suffice to get it into the release, and an unfortunate dynamic would be set up whereby each developer would vote for her own changes, yet would be reluctant to vote against others' changes, for fear of possible retaliation. To avoid this, the system should be arranged such that subgroups of developers must act in cooperation to get any change into the release. This not only means that more people review each change, it also makes any individual developer less hesitant to vote against a change, because she knows that no particular one among those who voted for it would take her vote against as a personal affront. The greater the number of people involved, the more the discussion becomes about the change and less about the individuals.

The system we use in the Subversion project seems to have struck a good balance, so I'll recommend it here. In order for a change to be applied to the release branch, at least three developers must vote in favor of it, and none against. A single "no" vote is enough to stop the change from being included; that is, a "no" vote in a release context is equivalent to a veto (see the section called “Vetoes”). Naturally, any such vote must be accompanied by a justification, and in theory the veto could be overridden if enough people feel it is unreasonable and force a special vote over it. In practice, this has never happened, and I don't expect that it ever will. People are conservative around releases anyway, and when someone feels strongly enough to veto the inclusion of a change, there's usually a good reason for it.

Because the release procedure is deliberately biased toward conservatism, the justifications offered for vetoes are sometimes procedural rather than technical. For example, a person may feel that a change is well-written and unlikely to cause any new bugs, but vote against its inclusion in a micro release simply because it's too big—perhaps it adds a new feature, or in some subtle way fails to fully follow the compatibility guidelines. I've occasionally even seen developers veto something because they simply had a gut feeling that the change needed more testing, even though they couldn't spot any bugs in it by inspection. People grumbled a little bit, but the vetoes stood and the change was not included in the release (I don't remember if any bugs were found in later testing or not, though).

Managing collaborative release stabilization

If your project chooses a change voting system, it is imperative that the physical mechanics of setting up ballots and casting votes be as convenient as possible. Although there is plenty of open source electronic voting software available, in practice the easiest thing to do is just to set up a text file in the release branch, called `STATUS` or `VOTES` or something like that. This file lists each proposed change—any developer can propose a change for inclusion—along with all the votes for and against it, plus any notes or comments. (Proposing a change doesn't necessarily mean voting for it, by the way, although the two often go together.) An entry in such a file might look like this:

```
* r2401 (issue #49)
  Prevent client/server handshake from happening twice.
  Justification:
    Avoids extra network turnaround; small change and easy to review.
  Notes:
    This was discussed in http://.../mailing-lists/message-7777.html
    and other messages in that thread.
  Votes:
    +1: jsmith, kimf
    -1: tmartin (breaks compatibility with some pre-1.0 servers;
                 admittedly, those servers are buggy, but why be
                 incompatible if we don't have to?)
```

In this case, the change acquired two positive votes, but was vetoed by tmartin, who gave the reason for the veto in a parenthetical note. The exact format of the entry doesn't matter; whatever your project settles on is fine—perhaps tmartin's explanation for the veto should go up in the "Notes:" section, or perhaps the change description should get a "Description:" header to match the other sections. The important thing is that all the information needed to evaluate the change be reachable, and that the mechanism for casting votes be as lightweight as possible. The proposed change is referred to by its revision number in the repository (in this case a single revision, r2401, although a proposed change could just as easily consist of multiple revisions). The revision is assumed to refer to a change made on the trunk; if the change were already on the release branch, there would be no need to vote on it. If your version control system doesn't have an obvious syntax for referring to individual changes, then the project should make one up. For voting to be practical, each change under consideration must be unambiguously identifiable.

Those proposing or voting for a change are responsible for making sure it applies cleanly to the release branch, that is, applies without conflicts (see *conflict*). If there are conflicts, then the entry should either point to an adjusted patch that does apply cleanly, or to a temporary branch that holds an adjusted version of the change, for example:

```
* r13222, r13223, r13232
  Rewrite libsvn_fs_fs's auto-merge algorithm
  Justification:
    unacceptable performance (>50 minutes for a small commit) in
    a repository with 300,000 revisions
  Branch:
    1.1.x-r13222@13517
  Votes:
    +1: epg, ghudson
```

That example is taken from real life; it comes from the `STATUS` file for the Subversion 1.1.4 release process. Notice how it uses the original revisions as canonical handles on the change, even though there is also a branch with a conflict-adjusted version of the change (the branch also combines the three trunk revisions into one, r13517, to make it easier to merge the change into the release, should it get approval). The original revisions are provided because they're still the easiest entity to review, since they have the original log messages. The temporary branch wouldn't have those log messages; in order to avoid duplication of information (see the section called “Singularity of information” in Chapter 3, *Technical Infrastructure*), the branch's log message for r13517 should simply say "Adjust r13222, r13223, and r13232 for backport to 1.1.x branch." All other information about the changes can be chased down at their original revisions.

Release manager

The actual process of merging (see *merge (a.k.a. port)*) approved changes into the release branch can be performed by any developer. There does not need to be one person whose job it is to merge changes; if there are a lot of changes, it can be better to spread the burden around.

However, although both voting and merging happen in a decentralized fashion, in practice there are usually one or two people driving the release process. This role is sometimes formally blessed as *release manager*, but it is quite different from a release owner (see the section called “Dictatorship by Release Owner” earlier in this chapter) who has final say over the changes. Release managers keep track of how many changes are currently under consideration, how many have been approved, how many seem likely to be approved, etc. If they sense that important changes are not getting enough attention, and might be left out of the release for lack of votes, they will gently nag other developers to review and vote. When a batch of changes are approved, these people will often take it upon themselves to merge them into the release branch; it's fine if others leave that task to them, as long as everyone understands that they are not obligated to do all the work unless they have explicitly committed to it. When the time comes to put the release out the door (see the section called “Testing and Releasing” later in this chapter), the release managers also take care of the logistics of creating the final release packages, collecting digital signatures, uploading the packages, and making the public announcement.

Packaging

The canonical form for distribution of free software is as source code. This is true regardless of whether the software normally runs in source form (i.e., can be interpreted, like Perl, Python, PHP, etc.) or needs to be compiled first (like C, C++, Java, etc.). With compiled software, most users will probably not compile the sources themselves, but will instead install from pre-built binary packages (see the section called “Binary Packages” later in this chapter). However, those binary packages are still derived from a master source distribution. The point of the source package is to unambiguously define the release. When the project distributes "Scanley 2.5.0", what it means, specifically, is "The tree of source code files that, when compiled (if necessary) and installed, produces Scanley 2.5.0."

There is a fairly strict standard for how source releases should look. One will occasionally see deviations from this standard, but they are the exception, not the rule. Unless there is a compelling reason to do otherwise, your project should follow this standard too.

Format

The source code should be shipped in the standard formats for transporting directory trees. For Unix and Unix-like operating systems, the convention is to use TAR format, compressed by **compress**, **gzip**, **bzip** or **bzip2**. For MS Windows, the standard method for distributing directory trees is *zip* format, which happens to do compression as well, so there is no need to compress the archive after creating it.

> **TAR Files**
>
> *TAR* stands for "Tape ARchive", because tar format represents a directory tree as a linear data stream, which makes it ideal for saving directory trees to tape. The same property also makes it the standard for distributing directory trees as a single file. Producing compressed tar files (or *tarballs*) is pretty easy. On some systems, the **tar** command can produce a compressed archive itself; on others, a separate compression program is used.

Name and Layout

The name of the package should consist of the software's name plus the release number, plus the format suffixes appropriate for the archive type. For example, Scanley 2.5.0, packaged for Unix using GNU Zip (gzip) compression, would look like this:

scanley-2.5.0.tar.gz

or for Windows using zip compression:

scanley-2.5.0.zip

Either of these archives, when unpacked, should create a single new directory tree named `scanley-2.5.0` in the current directory. Underneath the new directory, the source code should be arranged in a layout ready for compilation (if compilation is needed) and installation. In the top level of new directory tree, there should be a plain text `README` file explaining what the software does and what release this is, and giving pointers to other resources, such as the project's web site, other files of interest, etc. Among those other files should be an `INSTALL` file, sibling to the `README` file, giving instructions on how to build and install the software for all the operating systems it supports. As mentioned in the section called “How to Apply a License to Your Software” in Chapter 2, *Getting Started*, there should also be a `COPYING` or `LICENSE` file, giving the software's terms of distribution.

There should also be a `CHANGES` file (sometimes called `NEWS`), explaining what's new in this release. The `CHANGES` file accumulates changelists for all releases, in reverse chronological order, so that the list for this release appears at the top of the file. Completing that list is usually the last thing done on a stabilizing release branch; some projects write the list piecemeal as they're developing, others prefer to save it all up for the end and have one person write it, getting information by combing the version control logs. The list looks something like this:

```
Version 2.5.0
(20 December 2004, from /branches/2.5.x)
http://svn.scanley.org/repos/svn/tags/2.5.0/

 New features, enhancements:
    * Added regular expression queries (issue #53)
```

```
    * Added support for UTF-8 and UTF-16 documents
    * Documentation translated into Polish, Russian, Malagasy
    * ...

Bugfixes:
    * fixed reindexing bug (issue #945)
    * fixed some query bugs (issues #815, #1007, #1008)
    * ...
```

The list can be as long as necessary, but don't bother to include every little bugfix and feature enhancement. Its purpose is simply to give users an overview of what they would gain by upgrading to the new release. In fact, the changelist is customarily included in the announcement email (see the section called "Testing and Releasing" later in this chapter), so write it with that audience in mind.

CHANGES Versus ChangeLog

Traditionally, a file named *ChangeLog* lists every change ever made to a project—that is, every revision committed to the version control system. There are various formats for ChangeLog files; the details of the formats aren't important here, as they all contain the same information: the date of the change, its author, and a brief summary (or just the log message for that change).

A `CHANGES` file is different. It too is a list of changes, but only the ones thought important for a certain audience to see, and often with metadata like the exact date and author stripped off. To avoid confusion, don't use the terms interchangeably. Some projects use "NEWS" instead of "CHANGES"; although this avoids the potential for confusion with "ChangeLog", it is a bit of a misnomer, since the CHANGES file retains change information for all releases, and thus has a lot of old news in addition to the new news at the top.

ChangeLog files may be slowly disappearing anyway. They were helpful in the days when CVS was the only choice of version control system, because change data was not easy to extract from CVS. However, with more recent version control systems, the data that used to be kept in the ChangeLog can be requested from the version control repository at any time, making it pointless for the project to keep a static file containing that data—in fact, worse than pointless, since the ChangeLog would merely duplicate the log messages already stored in the repository.

The actual layout of the source code inside the tree should be the same as, or as similar as possible to, the source code layout one would get by checking out the project directly from its version control repository. Usually there are a few differences, for example because the package contains some generated files needed for configuration and compilation (see the section called "Compilation and Installation" later in this chapter), or because it includes third-party software that is not maintained by the project, but that is required and that users are not likely to already have. But even if the distributed tree corresponds exactly to some development tree in the version control repository, the distribution itself should not be a working copy (see *working copy*). The release is supposed to represent a static reference point—a particular, unchangeable configuration of source files. If it were a working copy, the danger would be that the user might update it, and afterward think that he still has the release when in fact he has something different.

Remember that the package is the same regardless of the packaging. The release—that is, the precise entity referred to when someone says "Scanley 2.5.0"—is the tree created by unpacking a zip file or tarball. So the project might offer all of these for download:

scanley-2.5.0.tar.bz2
scanley-2.5.0.tar.gz
scanley-2.5.0.zip

...but the source tree created by unpacking them must be the same. That source tree is the distribution; the form in which it is downloaded is merely a matter of convenience. Certain trivial differences between source packages are allowable: for example, in the Windows package, text files should have lines ending with CRLF (Carriage Return and Line Feed), while Unix packages should use just LF. The trees may be arranged slightly differently between source packages destined for different operating systems, too, if those operating systems require different sorts of layouts for compilation. However, these are all basically trivial transformations. The basic source files should be the same across all the packagings of a given release.

To capitalize or not to capitalize

When referring to a project by name, people generally capitalize it as a proper noun, and capitalize acronyms if there are any: "MySQL 5.0", "Scanley 2.5.0", etc. Whether this capitalization is reproduced in the package name is up to the project. Either `Scanley-2.5.0.tar.gz` or `scanley-2.5.0.tar.gz` would be fine, for example (I personally prefer the latter, because I don't like to make people hit the shift key, but plenty of projects ship capitalized packages). The important thing is that the directory created by unpacking the tarball use the same capitalization. There should be no surprises: the user must be able to predict with perfect accuracy the name of the directory that will be created when she unpacks a distribution.

Pre-releases

When shipping a pre-release or candidate release, the qualifier is truly a part of the release number, so include it in the name of the package's name. For example, the ordered sequence of alpha and beta releases given earlier in the section called “Release Number Components” would result in package names like this:

scanley-2.3.0-alpha1.tar.gz
scanley-2.3.0-alpha2.tar.gz
scanley-2.3.0-beta1.tar.gz
scanley-2.3.0-beta2.tar.gz
scanley-2.3.0-beta3.tar.gz
scanley-2.3.0.tar.gz

The first would unpack into a directory named `scanley-2.3.0-alpha1`, the second into `scanley-2.3.0-alpha2`, and so on.

Compilation and Installation

For software requiring compilation or installation from source, there are usually standard procedures that experienced users expect to be able to follow. For example, for programs written in C, C++, or certain other compiled languages, the standard under Unix-like systems is for the user to type:

```
$ ./configure
$ make
# make install
```

The first command autodetects as much about the environment as it can and prepares for the build process, the second command builds the software in place (but does not install it), and the last command installs it on the system. The first two commands are done as a regular user, the third as root. For more details about setting up this system, see the excellent *GNU Autoconf, Automake, and Libtool* book by Vaughan, Elliston, Tromey, and Taylor. It is published as treeware by New Riders, and its content is

also freely available online at http://sources.redhat.com/autobook/.

This is not the only standard, though it is one of the most widespread. The Ant (http://ant.apache.org/) build system is gaining popularity, especially with projects written in Java, and it has its own standard procedures for building and installing. Also, certain programming languages, such as Perl and Python, recommend that the same method be used for most programs written in that language (for example, Perl modules use the command **perl Makefile.pl**). If it's not obvious to you what the applicable standards are for your project, ask an experienced developer; you can safely assume that *some* standard applies, even if you don't know what it is at first.

Whatever the appropriate standards for you project are, don't deviate from them unless you absolutely must. Standard installation procedures are practically spinal reflexes for a lot of system administrators now. If they see familiar invocations documented in your project's `INSTALL` file, that instantly raises their faith that your project is generally aware of conventions, and that it is likely to have gotten other things right as well. Also, as discussed in the section called “Downloads” in Chapter 2, *Getting Started*, having a standard build procedure pleases potential developers.

On Windows, the standards for building and installing are a bit less settled. For projects requiring compilation, the general convention seems to be to ship a tree that can fit into the workspace/project model of the standard Microsoft development environments (Developer Studio, Visual Studio, VS.NET, MS-VC++, etc.). Depending on the nature of your software, it may be possible to offer a Unix-like build option on Windows via the Cygwin (http://www.cygwin.com/) environment. And of course, if you're using a language or programming framework that comes with its own build and install conventions—e.g., Perl or Python—you should simply use whatever the standard method is for that framework, whether on Windows, Unix, Mac OS X, or any other operating system.

Be willing to put in a lot of extra effort in order to make your project conform to the relevant build or installation standards. Building and installing is an entry point: it's okay for things to get harder after that, if they absolutely must, but it would be a shame for the user's or developer's very first interaction with the software to require unexpected steps.

Binary Packages

Although the formal release is a source code package, most users will install from binary packages, either provided by their operating system's software distribution mechanism, or obtained manually from the project web site or from some third party. Here "binary" doesn't necessarily mean "compiled"; it just means any pre-configured form of the package that allows a user to install it on his computer without going through the usual source-based build and install procedures. On RedHat GNU/Linux, it is the RPM system; on Debian GNU/Linux, it is the APT (`.deb`) system; on MS Windows, it's usually `.MSI` files or self-installing `.exe` files.

Whether these binary packages are assembled by people closely associated with the project, or by distant third parties, users are going to *treat* them as equivalent to the project's official releases, and will file issues in the project's bug tracker based on the behavior of the binary packages. Therefore, it is in the project's interest to provide packagers with clear guidelines, and work closely with them to see to it that what they produce represents the software fairly and accurately.

The main thing packagers need to know is that they should always base their binary packages on an official source release. Sometimes packagers are tempted to pull a later incarnation of the code from the repository, or include selected changes that were committed after the release was made, in order to provide users with certain bug fixes or other improvements. The packager thinks he is doing his users a favor by giving them the more recent code, but actually this practice can cause a great deal of confusion. Projects are prepared to receive reports of bugs found in released versions, and bugs found in recent trunk and major branch code (that is, found by people who deliberately run bleeding edge code). When a bug report comes in from these sources, the responder will often be able to confirm that the bug is known to be present in that snapshot, and perhaps that it has since been fixed and that the user should upgrade or wait for the next release. If it is a previously unknown bug, having the precise release makes it easier to reproduce and easier to categorize in the tracker.

Projects are not prepared, however, to receive bug reports based on unspecified intermediate or hybrid versions. Such bugs can be hard to reproduce; also, they may be due to unexpected interactions in isolated changes pulled in from later development, and thereby cause misbehaviors that the project's developers should not have to take the blame for. I have even seen dismayingly large amounts of time wasted because a bug was *absent* when it should have been present: someone was running a slightly patched up version, based on (but not identical to) an official release, and when the predicted bug did not happen, everyone had to dig around a lot to figure out why.

Still, there will sometimes be circumstances when a packager insists that modifications to the source release are necessary. Packagers should be encouraged to bring this up with the project's developers and describe their plans. They may get approval, but failing that, they will at least have notified the project of their intentions, so the project can watch out for unusual bug reports. The developers may respond by putting a disclaimer on the project's web site, and may ask that the packager do the same thing in the appropriate place, so that users of that binary package know what they are getting is not exactly the same as what the project officially released. There need be no animosity in such a situation, though unfortunately there often is. It's just that packagers have a slightly different set of goals from developers. The packagers mainly want the best out-of-the-box experience for their users. The developers want that too, of course, but they also need to ensure that they know what versions of the software are out there, so they can receive coherent bug reports and make compatibility guarantees. Sometimes these goals conflict. When they do, it's good to keep in mind that the project has no control over the packagers, and that the bonds of obligation run both ways. It's true that the project is doing the packagers a favor simply by producing the software. But the packagers are also doing the project a favor, by taking on a mostly unglamorous job in order to make the software more widely available, often by orders of magnitude. It's fine to disagree with packagers, but don't flame them; just try to work things out as best you can.

Testing and Releasing

Once the source tarball is produced from the stabilized release branch, the public part of the release process begins. But before the tarball is made available to the world at large, it should be tested and approved by some minimum number of developers, usually three or more. Approval is not simply a matter of inspecting the release for obvious flaws; ideally, the developers download the tarball, build and install it onto a clean system, run the regression test suite (see the section called “Automated testing”) in Chapter 8, *Managing Volunteers*, and do some manual testing. Assuming it passes these checks, as well as any other release checklist criteria the project may have, the developers then digitally sign the tarball using GnuPG (http://www.gnupg.org/), PGP (http://www.pgpi.org/), or some other program capable of producing PGP-compatible signatures.

In most projects, the developers just use their personal digital signatures, instead of a shared project key, and as many developers as want to may sign (i.e., there is a minimum number, but not a maximum). The more developers sign, the more testing the release undergoes, and also the greater the likelihood that a security-conscious user can find a digital trust path from herself to the tarball.

Once approved, the release (that is, all tarballs, zip files, and whatever other formats are being distributed) should be placed into the project's download area, accompanied by the digital signatures, and by MD5/SHA1 checksums (see http://en.wikipedia.org/wiki/Cryptographic_hash_function). There are various standards for doing this. One way is to accompany each released package with a file giving the corresponding digital signatures, and another file giving the checksum. For example, if one of the released packages is `scanley-2.5.0.tar.gz`, place in the same directory a file `scanley-2.5.0.tar.gz.asc` containing the digital signature for that tarball, another file `scanley-2.5.0.tar.gz.md5` containing its MD5 checksum, and optionally another, `scanley-2.5.0.tar.gz.sha1`, containing the SHA1 checksum. A different way to provide checking is to collect all the signatures for all the released packages into a single file, `scanley-2.5.0.sigs`; the same may be done with the checksums.

It doesn't really matter which way you do it. Just keep to a simple scheme, describe it clearly, and be consistent from release to release. The purpose of all this signing and checksumming is to give users a way to verify that the copy they receive has not been maliciously tampered with. Users are about to run

this code on their computers—if the code has been tampered with, an attacker could suddenly have a back door to all their data. See the section called “Security Releases” later in this chapter for more about paranoia.

Candidate Releases

For important releases containing many changes, many projects prefer to put out *release candidates* first, e.g., `scanley-2.5.0-beta1` before `scanley-2.5.0`. The purpose of a candidate is to subject the code to wide testing before blessing it as an official release. If problems are found, they are fixed on the release branch and a new candidate release is rolled out (`scanley-2.5.0-beta2`). The cycle continues until no unacceptable bugs are left, at which point the last candidate release becomes the official release—that is, the only difference between the last candidate release and the real release is the removal of the qualifier from the version number.

In most other respects, a candidate release should be treated the same as a real release. The *alpha*, *beta*, or *rc* qualifier is enough to warn conservative users to wait until the real release, and of course the announcement emails for the candidate releases should point out that their purpose is to solicit feedback. Other than that, give candidate releases the same amount of care as regular releases. After all, you want people to use the candidates, because exposure is the best way to uncover bugs, and also because you never know which candidate release will end up becoming the official release.

Announcing Releases

Announcing a release is like announcing any other event, and should use the procedures described in the section called “Publicity” in Chapter 6, *Communications*. There are a few specific things to do for releases, though.

Whenever you give the URL to the downloadable release tarball, make sure to also give the MD5/SHA1 checksums and pointers to the digital signatures file. Since the announcement happens in multiple forums (mailing list, news page, etc.), this means users can get the checksums from multiple sources, which gives the most security-conscious among them extra assurance that the checksums themselves have not been tampered with. Giving the link to the digital signature files multiple times doesn't make those signatures more secure, but it does reassure people (especially those who don't follow the project closely) that the project takes security seriously.

In the announcement email, and on news pages that contain more than just a blurb about the release, make sure to include the relevant portion of the CHANGES file, so people can see why it might be in their interests to upgrade. This is as important with candidate releases as with final releases; the presence of bugfixes and new features is important in tempting people to try out a candidate release.

Finally, don't forget to thank the development team, the testers, and all the people who took the time to file good bug reports. Don't single out anyone by name, though, unless there's someone who is individually responsible for a huge piece of work, the value of which is widely recognized by everyone in the project. Just be wary of sliding down the slippery slope of credit inflation (see the section called “Credit” in Chapter 8, *Managing Volunteers*).

Maintaining Multiple Release Lines

Most mature projects maintain multiple release lines in parallel. For example, after 1.0.0 comes out, that line should continue with micro (bugfix) releases 1.0.1, 1.0.2, etc., until the project explicitly decides to end the line. Note that merely releasing 1.1.0 is not sufficient reason to end the 1.0.x line. For example, some users make it a policy never to upgrade to the first release in a new minor or major series—they let others shake the bugs out of, say 1.1.0, and wait until 1.1.1. This isn't necessarily selfish (remember, they're forgoing the bugfixes and new features too); it's just that, for whatever reason, they've decided to be very careful with upgrades. Accordingly, if the project learns of a major bug in 1.0.3 right before it's about to release 1.1.0, it would be a bit severe to just put the bugfix in 1.1.0 and tell all the old 1.0.x

users they should upgrade. Why not release both 1.1.0 and 1.0.4, so everyone can be happy?

After the 1.1.x line is well under way, you can declare 1.0.x to be at *end of life*. This should be announced officially. The announcement could stand alone, or it could be mentioned as part of a 1.1.x release announcement; however you do it, users need to know that the old line is being phased out, so they can make upgrade decisions accordingly.

Some projects set a window of time during which they pledge to support the previous release line. In an open source context, "support" means accepting bug reports against that line, and making maintenance releases when significant bugs are found. Other projects don't give a definite amount of time, but watch incoming bug reports to gauge how many people are still using the older line. When the percentage drops below a certain point, they declare end of life for the line and stop supporting it.

For each release, make sure to have a *target version* or *target milestone* available in the bug tracker, so people filing bugs will be able to do so against the proper release. Don't forget to also have a target called "development" or "latest" for the most recent development sources, since some people—not only active developers—will often stay ahead of the official releases.

Security Releases

Most of the details of handling security bugs were covered in the section called “Announcing Security Vulnerabilities” in Chapter 6, *Communications*, but there are some special details to discuss for doing security releases.

A *security release* is a release made solely to close a security vulnerability. The code that fixes the bug cannot be made public until the release is available, which means not only that the fixes cannot be committed to the repository until the day of the release, but also that the release cannot be publicly tested before it goes out the door. Obviously, the developers can examine the fix among themselves, and test the release privately, but widespread real-world testing is not possible.

Because of this lack of testing, a security release should always consist of some existing release plus the fixes for the security bug, with *no other changes*. This is because the more changes you ship without testing, the more likely that one of them will cause a new bug, perhaps even a new security bug! This conservatism is also friendly to administrators who may need to deploy the security fix, but whose upgrade policy prefers that they not deploy any other changes at the same time.

Making a security release sometimes involves some minor deception. For example, the project may have been working on a 1.1.3 release, with certain bug fixes to 1.1.2 already publicly declared, when a security report comes in. Naturally, the developers cannot talk about the security problem until they make the fix available; until then, they must continue to talk publicly as though 1.1.3 will be what it's always been planned to be. But when 1.1.3 actually comes out, it will differ from 1.1.2 only in the security fixes, and all those other fixes will have been deferred to 1.1.4 (which, of course, will now *also* contain the security fix, as will all other future releases).

You could add an extra component to an existing release to indicate that it contains security changes only. For example, people would be able to tell just from the numbers that 1.1.2.1 is a security release against 1.1.2, and they would know that any release "higher" than that (e.g., 1.1.3, 1.2.0, etc.) contains the same security fixes. For those in the know, this system conveys a lot of information. On the other hand, for those not following the project closely, it can be a bit confusing to see a three-component release number most of the time with an occasional four-component one thrown in seemingly at random. Most projects I've looked at choose consistency and simply use the next regularly scheduled number for security releases, even when it means shifting other planned releases by one.

Releases and Daily Development

Maintaining parallel releases simultaneously has implications for how daily development is done. In particular, it makes practically mandatory a discipline that would be recommended anyway: have each

commit be a single logical change, and never mix unrelated changes in the same commit. If a change is too big or too disruptive to do in one commit, break it across N commits, where each commit is a well-partitioned subset of the overall change, and includes nothing unrelated to the overall change.

Here's an example of an ill-thought-out commit:

```
------------------------------------------------------------------------
r6228 | jrandom | 2004-06-30 22:13:07 -0500 (Wed, 30 Jun 2004) | 8 lines

Fix Issue #1729: Make indexing gracefully warn the user when a file
is changing as it is being indexed.

* ui/repl.py
  (ChangingFile): New exception class.
  (DoIndex): Handle new exception.

* indexer/index.py
  (FollowStream): Raise new exception if file changes during indexing.
  (BuildDir): Unrelatedly, remove some obsolete comments, reformat
  some code, and fix the error check when creating a directory.

Other unrelated cleanups:

* www/index.html: Fix some typos, set next release date.
------------------------------------------------------------------------
```

The problem with it becomes apparent as soon as someone needs to port the `BuildDir` error check fix over to a branch for an upcoming maintenance release. The porter doesn't want any of the other changes—for example, perhaps the fix to issue #1729 wasn't approved for the maintenance branch at all, and the `index.html` tweaks would simply be irrelevant there. But she cannot easily grab just the `BuildDir` change via the version control tool's merge functionality, because the version control system was told that that change is logically grouped with all these other unrelated things. In fact, the problem would become apparent even before the merge. Merely listing the change for voting would become problematic: instead of just giving the revision number, the proposer would have to make a special patch or change branch just to isolate the portion of the commit being proposed. That would be a lot of work for others to suffer through, and all because the original committer couldn't be bothered to break things into logical groups.

In fact, that commit really should have been *four* separate commits: one to fix issue #1729, another to remove obsolete comments and reformat code in `BuildDir`, another to fix the error check in `BuildDir`, and finally, one to tweak `index.html`. The third of those commits would be the one proposed for the maintenance release branch.

Of course, release stabilization is not the only reason why having each commit be one logical change is desirable. Psychologically, a semantically unified commit is easier to review, and easier to revert if necessary (in some version control systems, reversion is really a special kind of merge anyway). A little up-front discipline on everyone's part can save the project a lot of headache later.

Planning Releases

One area where open source projects have historically differed from proprietary projects is in release planning. Proprietary projects usually have firmer deadlines. Sometimes it's because customers were promised that an upgrade would be available by a certain date, because the new release needs to be coordinated with some other effort for marketing purposes, or because the venture capitalists who invested in the whole thing need to see some results before they put in any more funding. Free software projects, on the other hand, were until recently mostly motivated by amateurism in the most literal sense: they were written for the love of it. No one felt the need to ship before all the features were ready, and why

should they? It wasn't as if anyone's job was on the line.

Nowadays, many open source projects are funded by corporations, and are correspondingly more and more influenced by deadline-conscious corporate culture. This is in many ways a good thing, but it can cause conflicts between the priorities of those developers who are being paid and those who are volunteering their time. These conflicts often happen around the issue of when and how to schedule releases. The salaried developers who are under pressure will naturally want to just pick a date when the releases will occur, and have everyone's activities fall into line. But the volunteers may have other agendas—perhaps features they want to complete, or some testing they want to have done—that they feel the release should wait on.

There is no general solution to this problem except discussion and compromise, of course. But you can minimize the frequency and degree of friction caused, by decoupling the proposed *existence* of a given release from the date when it would go out the door. That is, try to steer discussion toward the subject of which releases the project will be making in the near- to medium-term future, and what features will be in them, without at first mentioning anything about dates, except for rough guesses with wide margins of error[1]. By nailing down feature sets early, you reduce the complexity of the discussion centered on any individual release, and therefore improve predictability. This also creates a kind of inertial bias against anyone who proposes to expand the definition of a release by adding new features or other complications. If the release's contents are fairly well defined, the onus is on the proposer to justify the expansion, even though the date of the release may not have been set yet.

In his multi-volume biography of Thomas Jefferson, *Jefferson and His Time*, Dumas Malone tells the story of how Jefferson handled the first meeting held to decide the organization of the future University of Virginia. The University had been Jefferson's idea in the first place, but (as is the case everywhere, not just in open source projects) many other parties had climbed on board quickly, each with their own interests and agendas. When they gathered at that first meeting to hash things out, Jefferson made sure to show up with meticulously prepared architectural drawings, detailed budgets for construction and operation, a proposed curriculum, and the names of specific faculty he wanted to import from Europe. No one else in the room was even remotely as prepared; the group essentially had to capitulate to Jefferson's vision, and the University was eventually founded more or less in accordance with his plans. The facts that construction went far over budget, and that many of his ideas did not, for various reasons, work out in the end, were all things Jefferson probably knew perfectly well would happen. His purpose was strategic: to show up at the meeting with something so substantive that everyone else would have to fall into the role of simply proposing modifications to it, so that the overall shape, and therefore schedule, of the project would be roughly as he wanted.

In the case of a free software project, there is no single "meeting", but instead a series of small proposals made mostly by means of the issue tracker. But if you have some credibility in the project to start with, and you start assigning various features, enhancements, and bugs to target releases in the issue tracker, according to some announced overall plan, people will mostly go along with you. Once you've got things laid out more or less as you want them, the conversations about actual release *dates* will go much more smoothly.

It is crucial, of course, to never present any individual decision as written in stone. In the comments associated with each assignment of an issue to a specific future release, invite discussion, dissent, and be genuinely willing to be persuaded whenever possible. Never exercise control merely for the sake of exercising control: the more deeply others participate in the release planning process (see the section called “Share Management Tasks as Well as Technical Tasks” in Chapter 8, *Managing Volunteers*), the easier it will be to persuade them to share your priorities on the issues that really count for you.

The other way the project can lower tensions around release planning is to make releases fairly often. When there's a long time between releases, the importance of any individual release is magnified in

[1]For an alternative approach, you may wish to read Martin Michlmayr's Ph.D. thesis *Quality Improvement in Volunteer Free and Open Source Software Projects: Exploring the Impact of Release Management* (http://www.cyrius.com/publications/michlmayr-phd.html). It is about using time-based release processes, as opposed to feature-based, in large free software projects. Michlmayr also gave a talk at Google on the subject, available on Google Video at http://video.google.com/videoplay?docid=-5503858974016723264.

everyone's minds; people are that much more crushed when their code doesn't make it in, because they know how long it might be until the next chance. Depending on the complexity of the release process and the nature of your project, somewhere between every three and six months is usually about the right gap between releases, though maintenance lines may put out micro releases a bit faster, if there is demand for them.